

Práctica 2. Conversión analógica-digital, PWM e interfaz gráfica

Requisitos y material

1. Computadora con Visual Studio Code y la extensión ESP-IDF instalada (v5 o superior)
2. Tarjeta Seeed Studio XIAO ESP32-S3 y cable USB-C
3. Pantalla OLED SSD1306 de 128×64 con interfaz I2C, de cuatro terminales: GND, VCC, SCL y SDA
4. Potenciómetro de tres terminales
5. Transistor MOSFET de canal N: 2N7000
6. Resistor de $1\text{ k}\Omega$
7. Resistor de $330\ \Omega$ y $220\ \Omega$ para variar el brillo del led
8. LED
9. Protoboard y cables de conexión
10. Solo para los que no tengan el material físico y lo hagan en simulación: cuenta en el simulador Velxio (velxio.dev) y el archivo del proyecto proporcionado por el profesor, practica2.vlx

Ejercicio 1. Circuito

La Figura 1 muestra las distintas funcionalidades de los puertos de la tarjeta ESP32-S3 Sense.

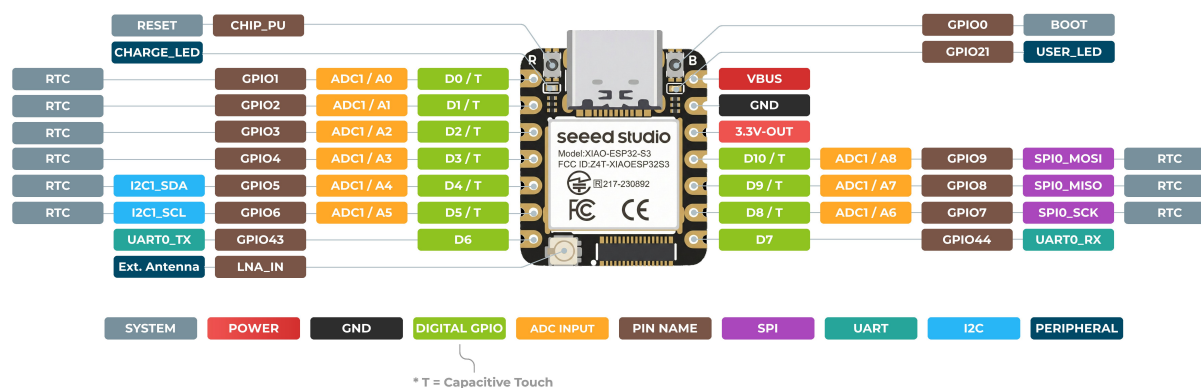


Figura 1. Terminales de la tarjeta ESP32-S3 Sense

Arma el circuito de la Figura 2 y verifica que las conexiones correspondan a las indicadas en la Tabla 1.

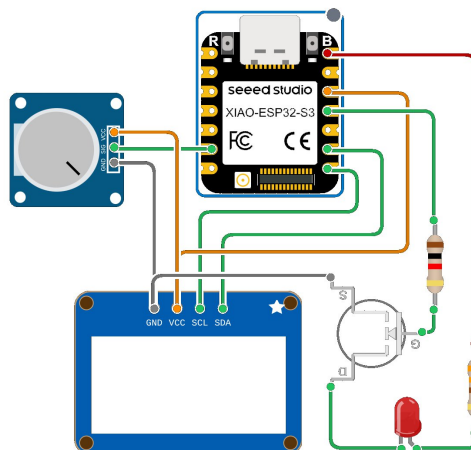


Figura 2. Circuito esquemático

Tabla 1. Conexiones del circuito

XIAO ESP32-S3	Se conecta a	Función
3V3	VCC del potenciómetro y VCC de la pantalla	Alimentación
GND	GND del potenciómetro, GND de la pantalla y fuente (S) del MOSFET	Referencia común
D5 (GPIO 6)	Salida del potenciómetro	Entrada analógica. ADC1, canal 5
D8 (GPIO 7)	SDA de la pantalla	Línea de datos I2C
D7 (GPIO 44)	SCL de la pantalla	Línea de reloj I2C
D10 (GPIO 9)	Resistor de 1 k Ω y de ahí a la compuerta (G) del MOSFET	Salida PWM
5V	Resistor de 330 Ω y de ahí al ánodo del LED	Alimentación del LED

El trabajo es en equipo. Quienes cuenten con el material físico deben armar el circuito sobre la protoboard. Quienes no lo tengan pueden apoyarse en el simulador Velxio. El simulador sirve únicamente para colaborar y verificar conexiones: **la entrega se realiza sobre el circuito físico.**

Notas:

- A) Identifica las terminales del MOSFET antes de conectarlo, tal como se muestra en la Fig. 3

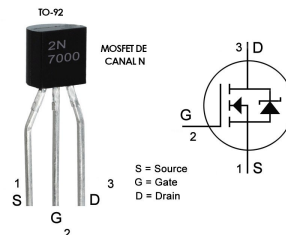


Figura 3. MOSFET 2N7000

Ejercicio 2. Programa

Para programar en la simulación en Velxio, descargar el archivo pr2.vlx, que incluye el esquemático y las librerías u8g2 de la pantalla OLED. Para programar en VSC, crear el proyecto desde cero y pegar la carpeta componentes del archivo components.zip, para trabajar con la librería u8g2 de la pantalla OLED.

El programa debe leer el potenciómetro con el ADC, ajustar el brillo del LED por medio de PWM y mostrar el valor de la lectura en la pantalla.

- La interfaz debe mostrar los tres elementos siguientes:
 - Una representación gráfica del valor del potenciómetro
 - El valor entregado por el ADC, de 0 a 4095
 - El porcentaje correspondiente, de 0 a 100 %
- La forma de la representación gráfica es libre. Puedes elegir alguna de las siguientes propuestas o plantear una propia:
 - Una barra horizontal o vertical que se llene conforme aumenta el valor
Una gráfica de x contra y que muestre la evolución de las lecturas recientes
 - Una carátula circular con una aguja que gire, como la de un reloj
 - Un arco graduado, al estilo de un tacómetro
- Reproduce el programa y verifica su funcionamiento.

Ejercicio 3. Separación en archivos (Tarea)

Una vez que el programa del ejercicio anterior funcione correctamente, guarda una copia del proyecto y reorganiza sobre ella el código en tres archivos, sin modificar su comportamiento.

1. `perifericos.h`, que contiene solamente declaraciones: las constantes definidas con `#define`, los prototipos de las funciones y, en su caso, la declaración de los handlers.
2. `perifericos.c`, que contiene las definiciones: el cuerpo de cada función y la definición de los handlers.
3. `main.c`, que únicamente incluye el encabezado con `#include "perifericos.h"` y contiene la función `app_main` con las llamadas.

Puntos a cuidar:

1. Investiga cómo exactamente hay que hacer los siguientes cambios:
 - Agregar el nuevo archivo `perifericos.c` en `CMakeLists.txt`. En `main/CMakeLists.txt` la línea queda como `idf_component_register(SRCS "main.c" "perifericos.c" INCLUDE_DIRS ".")`
 - Proteger el encabezado contra inclusiones repetidas con `#pragma once` en `perifericos.h`
 - Recuerda usar `extern` en `perifericos.h` para la declaración de variables
2. Compila y verifica que el comportamiento sea idéntico al del Ejercicio 2

Reporte

Elabora un reporte siguiendo el formato de la plantilla `Reporte_plantilla.docx` proporcionada por el profesor. Los siguientes puntos te pueden ayudar a redactar tu reporte.

1. Objetivos. Escribe los objetivos que identificaste en la práctica. Para ayudarte, puedes identificar el tema abordado y el aprendizaje obtenido.
2. Resultados y análisis. El análisis puede estar basado en las siguientes preguntas:
 - ¿Qué hacen las variables de configuración `ADC_BITWIDTH_12` y `ADC_ATTEN_DB_12`?
 - ¿Por cuáles otras variables se pueden cambiar y cómo afectan a la lectura del ADC?
 - ¿Por qué?"
 - ¿Qué pasa si cambio la frecuencia del PWM considerablemente?
 - ¿Cómo podrías aumentar la velocidad de actualización de la pantalla?
 - ¿Existe ruido en el valor obtenido del ADC?
3. Conclusiones. Puedes redactar tus conclusiones basado en las siguientes preguntas
 - ¿Qué dispositivos electrónicos pueden sustituir al potenciómetro y para qué?
 - ¿Qué dispositivos electrónicos pueden sustituir al led y para qué?
 - ¿Cómo puedes reducir el ruido de la señal recibida por el ADC?
 - ¿Qué ventajas tiene separar el programa en un archivo de encabezado y uno de implementación, frente a tenerlo todo en `main.c`?